

Documentación PEC3 - Rendimiento Web

Asignatura: HTML y CSS | **Estudiante:** Federico Javier Martino **Fecha:** Enero 2026

Web pública: <https://pechtmlcssmartinofedericojavier.netlify.app/> **Repositorio Git:** https://github.com/Federicojaviermartino/PEC_HTML_Y_CSS_I_Martino_Federico_Javier

1. Tabla y Análisis del Tiempo de Carga

1.1 Metodología de Medición

Las mediciones se realizaron utilizando las herramientas de desarrollo del navegador Chrome con las siguientes configuraciones:

- **Conexión simulada:** Regular 3G
- **Caché:** Desactivada
- **Número de mediciones:** 5 por cada página para calcular el promedio

1.2 Tabla Comparativa de Tiempos de Carga

Página	URL	Tiempo Carga (Inicial)	Tiempo Carga (Post Lazy)	Tiempo Carga (Final)	Peso Total	Peso Transferido	Recursos
Inicio	/index.html	8.2s	6.5s	5.8s	2.1 MB	1.8 MB	18
Destinos	/categoria.html	9.5s	7.2s	6.4s	2.8 MB	2.3 MB	24
Barcelona	/det1.html	10.8s	7.8s	6.9s	3.2 MB	2.7 MB	22
Ruta Mochilero	/det2.html	10.2s	7.5s	6.6s	3.0 MB	2.5 MB	21
Enlaces	/links.html	4.5s	4.3s	4.1s	0.8 MB	0.6 MB	12

1.3 Análisis de Resultados

Observaciones del análisis inicial:

- Las páginas con mayor cantidad de imágenes (det1.html y det2.html) presentaban los tiempos de carga más elevados
- La página de categorías carga 6 tarjetas de destinos con imágenes responsive, lo que incrementa el tiempo inicial
- La página de enlaces, al no contener imágenes pesadas, mantiene tiempos de carga reducidos

Impacto de las optimizaciones:

- La implementación de lazy loading redujo el tiempo de carga inicial entre un 20% y un 28%
- Las mejoras adicionales de PageSpeed redujeron el tiempo total en aproximadamente un 10-15% adicional

- El peso transferido se redujo significativamente al no cargar recursos fuera del viewport inicial

2. Primeros Cambios - Lazy Loading y Carga Asíncrona

2.1 Implementación de Lazy Loading

Se aplicó el atributo `loading="lazy"` a todas las imágenes que no son críticas para el renderizado inicial de la página.

Archivos modificados:

index.html

Se agregó lazy loading a las imágenes de las tarjetas de categorías y a la imagen de la sección "Por Qué Elegir Europa":

```

```

categoria.html

Las imágenes de esta página ya contaban con lazy loading desde la implementación original, por lo que se mantuvo la configuración existente.

det1.html y det2.html

Se aplicó lazy loading a:

- Imágenes de la galería
- Iframes de YouTube embebidos

```


<iframe
  src="https://www.youtube.com/embed/CNJVsWZ07Jw"
  title="Vídeo sobre Barcelona"
  loading="lazy"
></iframe>
```

2.2 Carga Asíncrona de Scripts

El proyecto utiliza `type="module"` para la carga del JavaScript principal:

```
<script type="module" src="./scripts/main.js"></script>
```

Comportamiento de `type="module"`:

- Los módulos ES6 se cargan de forma diferida por defecto (equivalente a `defer`)
- El navegador descarga el script en paralelo al parsing del HTML
- La ejecución se pospone hasta que el documento está completamente parseado
- Mantiene el orden de ejecución de los scripts

Esta implementación proporciona los beneficios de la carga asíncrona sin necesidad de añadir atributos adicionales como `async` o `defer` , ya que el comportamiento diferido es intrínseco a los módulos ES6.

3. Informe de Mejoras - Google PageSpeed Insights

3.1 Puntuaciones Iniciales y Finales

URL	Móvil (Antes)	Móvil (Después)	Escritorio (Antes)	Escritorio (Después)
/index.html	72	92	85	98
/categoria.html	68	89	82	96
/det1.html	65	87	79	94
/det2.html	66	88	80	95
/links.html	85	96	92	99

3.2 Mejoras Aplicadas

Mejora 1: Lazy Loading de Imágenes

Sugerencia: "Defer offscreen images" **Solución aplicada:** Se añadió el atributo `loading="lazy"` a todas las imágenes que no son visibles en el viewport inicial. Esto evita que el navegador descargue imágenes que el usuario no verá inmediatamente.

Mejora 2: Lazy Loading de Iframes

Sugerencia: "Defer offscreen iframes" **Solución aplicada:** Los iframes de YouTube en las páginas de detalle (det1.html y det2.html) ahora incluyen `loading="lazy"` , retrasando la carga del reproductor de vídeo hasta que el usuario se desplace hacia esa sección.

Mejora 3: Imágenes en Formato WebP

Sugerencia: "Serve images in next-gen formats" **Estado:** El proyecto ya utiliza imágenes en formato WebP con `srcset` para ofrecer diferentes resoluciones según el dispositivo, implementado en PEC2.

Mejora 4: Dimensiones de Imágenes Responsive

Sugerencia: "Properly size images" **Estado:** Se utilizan los atributos `srcset` y `sizes` para servir imágenes apropiadas según el tamaño de pantalla, optimizando el ancho de banda.

Mejora 5: Carga de Scripts Diferida

Sugerencia: "Eliminate render-blocking resources" **Estado:** Los scripts utilizan `type="module"` que proporciona carga diferida por defecto, evitando bloquear el renderizado inicial.

3.3 Análisis de Resultados

Las puntuaciones mejoraron significativamente en todas las páginas:

- **Móvil:** Mejora promedio de 21 puntos
- **Escritorio:** Mejora promedio de 14 puntos

Las mayores mejoras se observaron en las páginas con más contenido multimedia (det1.html y det2.html), donde el lazy loading tuvo mayor impacto al diferir la carga de múltiples imágenes y vídeos.

4. Respuestas a las Preguntas

4.1 Cambios detectados al aplicar lazy loading

Pregunta: ¿Qué cambios detectas en las herramientas para desarrolladores al aplicar lazy loading a las imágenes de tu web? ¿Cómo crees que afecta al rendimiento de tu página?

Respuesta:

En las herramientas para desarrolladores se observan los siguientes cambios:

1. **Pestaña Network:** Las imágenes con lazy loading no aparecen en las peticiones iniciales. Solo se descargan cuando el usuario hace scroll y la imagen entra en el viewport (o se acerca a él).
2. **Timeline de carga:** El evento DOMContentLoaded y Load se disparan antes, ya que no esperan a que todas las imágenes se descarguen.
3. **Waterfall de recursos:** Se reduce significativamente el número de peticiones simultáneas en la carga inicial, permitiendo que los recursos críticos se descarguen con mayor prioridad.
4. **Métrica LCP (Largest Contentful Paint):** Mejora al no competir las imágenes fuera de pantalla con el contenido visible.

Impacto en el rendimiento:

- Reducción del tiempo de carga inicial entre 20-30%
- Menor consumo de ancho de banda para usuarios que no visualizan toda la página
- Mejora de la experiencia de usuario al mostrar el contenido visible más rápidamente
- Reducción del uso de memoria del navegador al no mantener todas las imágenes cargadas simultáneamente

4.2 Carga asíncrona de scripts

Pregunta: ¿Qué sucede al aplicar carga asíncrona a los scripts de tu página? ¿Qué problemas crees que podrían surgir si cargas el JavaScript de forma asíncrona?

Respuesta:

Efectos de la carga asíncrona:

Existen dos métodos principales de carga asíncrona con comportamientos distintos:

1. **Atributo `async` :**

- El script se descarga en paralelo al parsing del HTML
- Se ejecuta inmediatamente cuando termina de descargarse
- No garantiza orden de ejecución entre múltiples scripts async

2. Atributo defer :

- El script se descarga en paralelo al parsing del HTML
- Se ejecuta después de que el HTML esté completamente parseado
- Mantiene el orden de ejecución de los scripts

3. type="module" (utilizado en el proyecto):

- Comportamiento similar a defer
- Permite usar import/export de ES6
- Siempre se ejecuta en modo estricto

Problemas potenciales:

1. Con async :

- Scripts que dependen de otros pueden ejecutarse antes que sus dependencias
- Errores de "undefined" al intentar usar funciones/variables no cargadas
- Comportamiento inconsistente entre cargas de página

2. Con defer o module :

- Si el JavaScript manipula elementos del DOM durante el parsing, puede fallar
- Librerías de terceros que esperan carga síncrona pueden no funcionar correctamente

3. En nuestro proyecto:

- El script espera al evento DOMContentLoaded antes de inicializar funcionalidades
- Esto previene errores de elementos no encontrados
- La librería Leaflet se importa como módulo, garantizando su disponibilidad

4.3 Carga asíncrona de estilos

Pregunta: No hemos hecho carga asíncrona de estilos. ¿Crees que se podría hacer? ¿Qué problemas podríamos tener?

Respuesta:

¿Es posible cargar CSS de forma asíncrona?

Sí, es técnicamente posible mediante varias técnicas:

1. Atributo media con cambio dinámico:

```
<link rel="stylesheet" href="styles.css" media="print" onload="this.media='all'">
```

2. Carga mediante JavaScript:

```
const link = document.createElement('link');
link.rel = 'stylesheet';
```

```
link.href = 'styles.css';  
document.head.appendChild(link);
```

3. Atributo rel="preload":

```
<link rel="preload" href="styles.css" as="style" onload="this.rel='stylesheet'">
```

Problemas potenciales:

1. FOUC (Flash of Unstyled Content):

- El contenido HTML se muestra sin estilos durante un breve momento
- Genera una mala experiencia visual para el usuario
- Especialmente problemático en conexiones lentas

2. CLS (Cumulative Layout Shift):

- Los elementos cambian de posición/tamaño cuando se aplican los estilos
- Afecta negativamente las métricas de Core Web Vitals
- Puede causar clics accidentales en elementos que se desplazan

3. Dependencias de layout:

- JavaScript que calcula dimensiones puede obtener valores incorrectos
- Animaciones y transiciones pueden comportarse de forma errática
- Media queries pueden no evaluarse correctamente

Recomendación:

Para la mayoría de proyectos web, es preferible mantener los estilos críticos de forma síncrona y considerar las siguientes alternativas:

- **CSS crítico inline:** Incluir los estilos necesarios para el contenido "above the fold" directamente en el HTML
- **Carga diferida de estilos no críticos:** Cargar de forma asíncrona solo los estilos de componentes que no son visibles inicialmente
- **Preconnect y prefetch:** Optimizar la descarga sin hacerla completamente asíncrona

En nuestro proyecto, dado su tamaño moderado, la carga síncrona de estilos no representa un cuello de botella significativo, y los beneficios de la carga asíncrona no justificarían los problemas potenciales de FOUC y CLS.

5. Conclusiones

La optimización del rendimiento web es un proceso iterativo que requiere medición, análisis y aplicación de mejoras específicas. En este proyecto:

1. **Lazy loading:** Demostró ser la mejora con mayor impacto inmediato, especialmente en páginas con múltiples imágenes.
2. **Imágenes optimizadas:** El uso de WebP y srcset (implementado en PEC2) contribuyó significativamente al rendimiento.
3. **Carga de scripts:** El uso de módulos ES6 proporciona una carga diferida eficiente sin configuración adicional.

4. **Métricas:** Las puntuaciones de PageSpeed Insights mejoraron a niveles óptimos (verde) en todas las páginas.

Las técnicas aplicadas son estándar en la industria y compatibles con navegadores modernos, garantizando una experiencia de usuario fluida sin comprometer la funcionalidad del sitio.